

AWS GenAI 課程 · 第 11 單元

DNS 與內容分發

盧建霖 • 2026-10-01

改了 **index.html**，使用者還在看舊版。
問題不在伺服器，在**門口那份快取**。

Agenda

- 01 DNS 記錄與路由策略
- 02 健康檢查與故障切換
- 03 CloudFront 快取與 OAC
- 04 加速連線與 AI 應用的邊界
- 05 實作：S3 加 CloudFront 鎖桶

TODAY'S GOAL

替自己的網域接上
CloudFront，桶維持不
公開，改檔後知道怎麼
讓全世界換新版。

前兩段講名字怎麼指到位址，中間兩
段講內容與連線怎麼加速，最後親手
建一次。

先定義名詞，後面才好講

Route 53 是 AWS 的 DNS 服務，CloudFront 是 AWS 的內容分發網路；這六個詞今天會反覆出現

網域名稱系統 **DNS**

Domain Name System，把網域名稱翻成 IP 位址的全球查號台。

託管區域 **hosted zone**

Route 53 裡一個網域底下所有記錄的容器，查詢的最後一站。

存活時間 **TTL**

time to live，答案可以被快取幾秒；**DNS** 與 **CloudFront** 都用它。

別名記錄 **Alias**

Route 53 專屬的擴充，直接回 **AWS** 資源當下的位址。

邊緣位置 **edge location**

離使用者最近的 **AWS** 快取機房，又叫 **POP**。

來源 **origin**

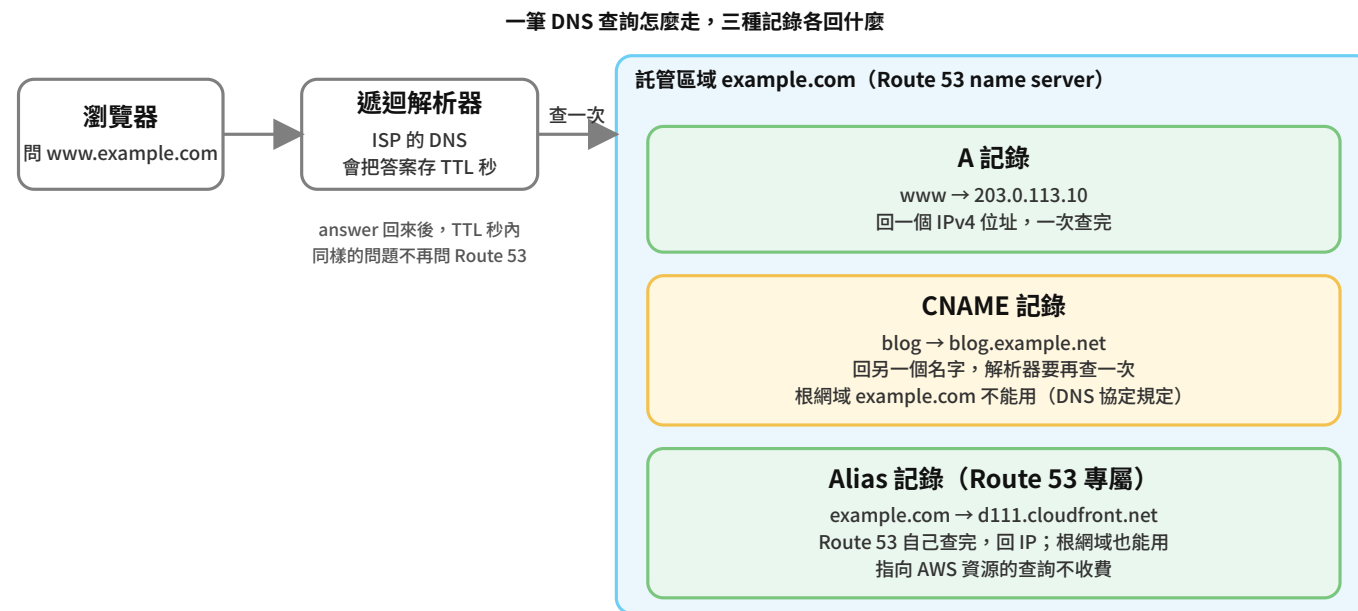
放原始檔案的地方，例如 **S3** 桶或任何 **HTTP** 伺服器。

名字怎麼變成位址

- 一、查詢一路轉問，託管區域是最後一站
- 二、IP 會變的 **AWS** 資源，接名字用 **Alias**
- 三、同名多筆記錄，路由策略決定回哪一筆

一筆查詢一路轉問，問到 Route 53 才有答案

解析器先問根、再問 .com，每一站只告訴你下一站是誰；拿到 IP 後網頁請求直接連過去



- 1 遞迴解析器 (DNS resolver)：替你一路問到底、再把答案記起來的中間人
- 2 右邊三張卡是三種記錄：綠色一次查完，黃色要再查一次
- 3 拿到 IP 後，網頁請求直連 ALB，不再經過 Route 53

TTL 長省錢，改 IP 生效就慢

TTL 填 300：每個解析器從拿到答案起，五分鐘內都用手上這份；各自倒數，不是全世界一起

1 改設定前，先把 TTL 縮到 300 秒

官方建議；確認沒問題再拉長。

2 順序要對：先縮 TTL，等舊答案陸續到期

之後再改 IP，變更才比較快被看到。

3 託管區域自帶 NS 與 SOA 兩筆

NS 寫網域歸哪幾台伺服器管；SOA 是區域的基本資料。

網域在別家買的：自己建同名託管區域，到註冊商把 **name server** 換成 Route 53 給的四個。

三種記錄回三種東西

CNAME (Canonical Name) 把名字指到另一個名字；IPv6 版的 A 記錄叫 AAAA

記錄	你填什麼	解析器拿到什麼	根網域	查詢收費
A	一個 IPv4 位址	IP，一次查完	能	收
CNAME	另一個名字	名字，還要再查一次	不能	收；指到同區域的名字算兩次
Alias	AWS 資源或同區域另一筆記錄	資源當下的位址	能	指向 AWS 資源免費

Alias 是掛在既有型別上的開關：選 A 或 AAAA 再打開它；dig 看到的就是你選的型別。

根網域要指到 CloudFront：答案是 Alias

根網域 (zone apex)：跟託管區域同名的那個名字，例如 example.com；DNS 協定規定 CNAME 不能放在這裡

你想做的事	天真做法	結果	正解
www 指到 CloudFront	www CNAME d111.cloudfront.net	建得起來，但收查詢 費、多查一次	www Alias 到 distribution
example.com 指到 CloudFront	example.com CNAME d111...	建不起來	example.com Alias 到 distribution
example.com 指到 www	example.com CNAME www	建不起來	Alias 到 www 那筆，那筆不 能是 CNAME

IP 會變的資源用 A 記錄指，位址一跑就斷；CNAME 又碰不了根網域，所以 Alias 是標準做法。

動手做：用 dig 看一個真實網域的 CNAME 與 TTL (1/2)

第 2 步那個 CNAME 就是「回一個名字，解析器再查一次」，第 3 步那個多半會變小的數字就是 TTL 在倒數；官方文件說 dig 看不出 Alias，Alias 會顯示成建它時選的型別

小步	做什麼	做完應該看到
1	Region 選 Asia Pacific (Tokyo)，主控台上方搜尋列旁邊點終端機圖示，打開 CloudShell (主控台內建的瀏覽器終端機，不用在自己電腦裝 CLI)	一個 Bash 提示字元，前面是 cloudshell-user
2	打 <code>dig +noall +answer www.amazon.com</code>	好幾行輸出；至少一行第四欄是 CNAME、後面接另一個名字，最後一行第四欄是 A、後面是 IP (這是講師 2026-09-24 實測的樣子，官方文件沒寫，大網站的設定會變)。dig 這個工具預設會把 CNAME 與順著它查到的 A 一起列出來，所以你一次就看到整條鏈 (講師補充，這是 dig 的行為，AWS 文件沒寫)
3	看每一行的第二欄那個數字	那是剩餘 TTL 秒數；再打一次同樣的指令，數字通常會變小；偶爾反而跳回一個大數字，最可能的原因是剛好到期、解析器重新問了一次，從頭倒數 (這是講師經驗，官方文件只講到期會重查)
4	打 <code>dig +noall +answer amazon.com</code>	只有 A 記錄，沒有 CNAME：根網域要指到 IPv4 只能用 A (IPv6 用 AAAA) 或 Alias

HANDS-ON · 學員跟做 · 約 6 分鐘 · 0 元，只查別人的網域，不建任何資源

動手做：用 dig 看一個真實網域的 CNAME 與 TTL (2/2)

第 2 步那個 CNAME 就是「回一個名字，解析器再查一次」，第 3 步那個多半會變小的數字就是 TTL 在倒數；官方文件說 dig 看不出 Alias，Alias 會顯示成建它時選的型別

小步	做什麼	做完應該看到
5	打 <code>dig +noall +answer d1.awsstatic.com</code>	一串 IP，中間可能有 CNAME 指到 <code>cloudfront.net</code> 結尾的名字，那就是第 4 課要講的 CloudFront

QUIZ 01 · CLF-C02

哪一個 **AWS** 服務提供 **DNS** 解析功能？

A

Amazon CloudFront

B

Amazon VPC

C

Amazon Route 53

D

AWS Direct Connect

想 30 秒，等一下翻下一頁對答案。

QUIZ 01 · CLF-C02 · ANSWER

哪一個 **AWS** 服務提供 **DNS** 解析功能？

A

Amazon CloudFront

B

Amazon VPC

C

Amazon Route 53

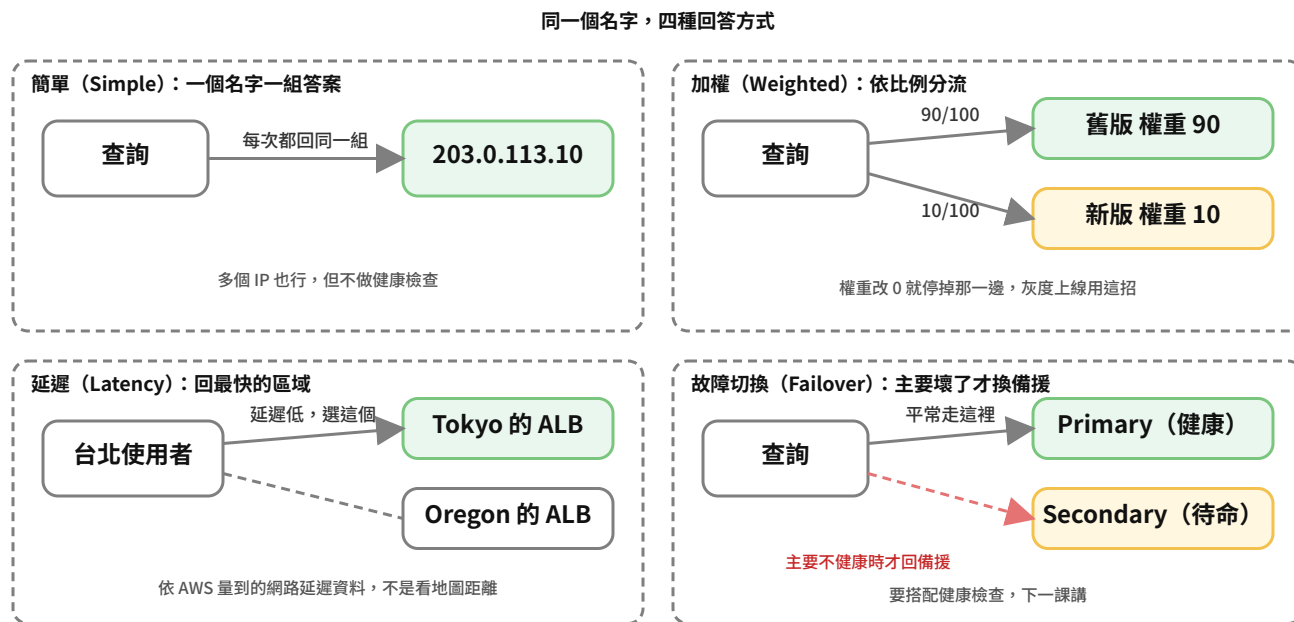
D

AWS Direct Connect

C。「提供 DNS 解析功能」的服務是 Route 53；CloudFront 常被拿來誘答，它靠 DNS 把使用者導到邊緣位置，自己不是 DNS 服務。VPC 內建的 DNS 背後是 Route 53 Resolver，但考試要的是服務名稱。

同一個名字，四種挑法

路由策略（routing policy）：建記錄時選的規則，決定同名多筆記錄時 Route 53 回哪一筆



- 1 左上簡單、右上加權、左下延遲、右下故障切換
- 2 綠色是這次被回出去的答案，虛線是沒被選到的
- 3 分流這件事，在 DNS 這一層就開始了

一筆 **A** 記錄填兩個 **IP**，夠不夠？

× 大家通常會想

東京、奧勒岡兩個 IP 填同一筆，Route 53 會幫我分流。

簡單路由把全部的值隨機排序回給解析器，由瀏覽器自己挑一個。

✓ 實際上

它不判斷誰離使用者近，也不做健康檢查。

東京那台掛了照樣回給你；要挑，就建同名多筆記錄配路由策略。

四種要會用：平常怎麼分，出事怎麼換

策略	挑法	每筆多填	典型場景
簡單 Simple	一個名字一筆記錄，照填的值全部回	沒有	單一伺服器
加權 Weighted	依權重比例回，權重 0-255	Weight、Record ID	新版先給 10% 流量
延遲 Latency	回 AWS 量到延遲最低的區域	Region	多區域部署
故障切換 Failover	主要健康回主要，不健康才回次要	Primary 或 Secondary、健康檢查	主站加維修頁

前三種是平常怎麼分，故障切換是出事怎麼換；兩邊都不健康時，故障切換回主要。

加權看比例，延遲看網路不看地圖

加權是 DNS 回答的比例；解析器會快取，實際請求數只是大致照這個比例走

1 權重 1 對 255：一筆拿 1/256 的回答

官方的例子；比例是「這筆權重除以總權重」。

2 要停掉一邊，權重改 0，不用刪記錄

灰度上線：新版先給 1，錯誤率沒變差再調上去。

3 延遲資料會隨時間變

倫敦的使用者這週到奧勒岡，下週可能變新加坡。

延遲資料只涵蓋 AWS 區域；資源不在 AWS 區域裡，就算同一個城市，量到的延遲也可能差很多。

另外四種，認得名字就好

策略	一句話	什麼時候想到它
地理位置 Geolocation	看使用者在哪個洲、國家	內容只在有授權的地區提供
地理鄰近 Geoproximity	看人與資源的距離， bias 可放大縮小範圍	手動把流量從 A 區推一些到 B 區
多值回答 Multivalue	最多回八筆健康的記錄	沒有 ALB，想用 DNS 做一點分流
IP 位址 IP-based	你上傳來源 IP 段對端點的表	特定 ISP（網路服務商）導到特定端點

地理位置記得多建一筆 **Default**：IP 對不到地區又沒有預設，Route 53 回「沒有答案」。

查詢按百萬次計，十萬次是美分

每月前十億次的單價；超過十億次的部分單價減半；一個月十萬次標準查詢是 0.04 美元

策略	每百萬次查詢
標準：簡單、加權、故障切換、多值回答	0.40 美元
延遲	0.60 美元
地理位置、地理鄰近	0.70 美元
IP 位址	0.80 美元

兩個例外不算錢：指向 AWS 資源的 Alias，以及私有託管區域的查詢。

HANDS-ON · 講師示範 · 約 8 分鐘 · 接近 0 元（講師估算）：託管區域建立後 12 小時內刪掉不收區域費；公有區域的查詢照計價

動手做：看 **Create record** 表單的 **Routing policy** 下拉選單長什麼樣 (1/2)

第 4 步是這一課的全部：策略不同，每筆記錄要多填的參數就不同，Weight 是比例、Region 是延遲資料要查的區域、Failover record type 是主次；表單本身就是這張總結表

小步	做什麼	做完應該看到
1	Route 53 主控台 → Hosted zones → Create hosted zone ； Domain name 填一個不存在的示範名字，例如 a07-demo.test ； Type 留 Public hosted zone ；按 Create hosted zone	進到區域頁，Records 裡已經有 NS 與 SOA 兩筆，你什麼都沒建
2	按 Create record	表單出現 Record name 、Record type 、Value 、TTL (seconds) 、Routing policy 這幾個欄位
3	點開 Routing policy 下拉選單	八個選項：Simple routing 、Weighted 、Geolocation 、Latency 、Failover 、Multivalue answer 、IP-based 、Geoproximity
4	依序選 Weighted 、Latency 、Failover ，每選一個看表單多出什麼欄位	Weighted 多出 Weight 與 Record ID ；Latency 多出 Region ；Failover 多出 Failover record type （Primary / Secondary）與 Health check

HANDS-ON · 講師示範 · 約 8 分鐘 · 接近 0 元（講師估算）：託管區域建立後 12 小時內刪掉不收區域費；公有區域的查詢照計價

動手做：看 **Create record** 表單的 **Routing policy** 下拉選單長什麼樣 (2/2)

第 4 步是這一課的全部：策略不同，每筆記錄要多填的參數就不同，Weight 是比例、Region 是延遲資料要查的區域、Failover record type 是主次；表單本身就是這張總結表

小步	做什麼	做完應該看到
5	按 Cancel 離開表單	回到區域頁， Records 仍然只有 NS 與 SOA
6	清理：回 Hosted zones ，勾 a07-demo.test → Delete ；依畫面指示輸入確認字串	列表沒有它；只剩 NS 與 SOA 的區域可以直接刪

QUIZ 02 · CLF-C02

Amazon CloudFront 提供什麼功能？

- A** 從單一統一介面自動調整所有資源以支援應用程式
- B** 安全地將資料、影片、應用程式和 API 以低延遲傳送給全球使用者
- C** 能夠透過多種路由類型直接管理全球流量，包括基於延遲的路由、地理 DNS、地理鄰近和加權輪詢
- D** 自動將進來的應用程式流量分配到多個目標，例如 Amazon EC2 實例、容器、IP 位址和AWS Lambda 函數

想 30 秒，等一下翻下一頁對答案。

QUIZ 02 · CLF-C02 · ANSWER

Amazon CloudFront 提供什麼功能？

- ☐ A 從單一統一介面自動調整所有資源以支援應用程式
- ☒ B 安全地將資料、影片、應用程式和 API 以低延遲傳送給全球使用者
- ☐ C 能夠透過多種路由類型直接管理全球流量，包括基於延遲的路由、地理 DNS、地理鄰近和加權輪詢
- ☐ D 自動將進來的應用程式流量分配到多個目標，例如 Amazon EC2 實例、容器、IP 位址和 AWS Lambda 函數

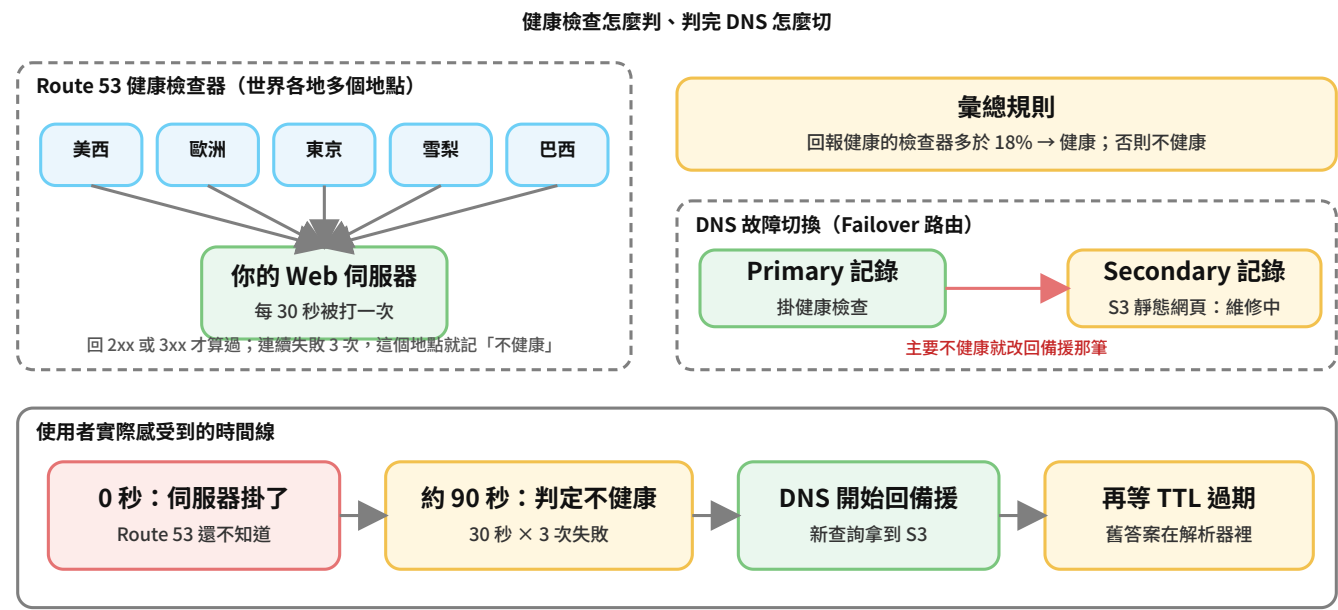
B。CloudFront 以低延遲把資料、影片、應用程式與 API 安全地送給全球使用者。C 那串（延遲、地理 DNS、地理鄰近、加權輪詢）是 Route 53 的路由策略；A 是集中管理資源的工具，D 是 ELB。

壞了，自動換一條路

- 一、很多檢查器投票，超過 **18%** 說好就算健康
- 二、故障切換是路由策略加健康檢查的組合
- 三、最壞斷線時間：判定時間加上舊答案的 **TTL**

伺服器掛了，使用者要等兩段時間

健康檢查 (health check)：Route 53 從世界各地定期敲你的伺服器，判定它活著沒有



30 秒與 3 次是範例設定，主控台的預設值以畫面為準

- 1 左上：各地檢查器同時敲，單一地區的網路問題不會誤判
- 2 右上：彙總規則，加上 Primary 與 Secondary 那一對記錄
- 3 最後一格：DNS 換了答案，還要等舊答案的 TTL 過期

一個檢查器怎麼評分：先敲門，再看回話

Failure threshold（失敗門檻）：連續失敗幾次，這個檢查器才把它記成不健康

1 HTTP 或 HTTPS：4 秒內連上，2 秒內收到 2xx 或 3xx

TCP 檢查只看連不連得上，10 秒內連上就算。

2 間隔選每 30 秒或每 10 秒

連續失敗到門檻次數，才翻成不健康。

3 回報健康的檢查器多於 18%，就算健康

真的掛了，各地會一起連不上，比例掉到接近 0。

100 個檢查器裡 19 個說正常，照規則還是健康；18% 算的是檢查器數目，不是地區數。

HTTPS 檢查說健康，憑證可能已過期

× 大家通常會想

HTTPS 健康檢查會順便驗證憑證。

憑證過期，網站在瀏覽器打不開，檢查卻一直顯示健康。

✓ 實際上

HTTPS 檢查不驗證憑證，只看連上與 2xx 或 3xx。

憑證到期要另外監控；剛建好的檢查，資料不夠前也算健康。

三種健康檢查：敲端點、數別人、看指標

CloudWatch：AWS 的監控服務，把 CPU 幾 % 這類數字隨時間記成一條指標（metric）

類型	監控什麼	什麼時候用
監控端點	一個 IP 或網域，用 HTTP、HTTPS、TCP 敲	最常見，敲網站或 API
計算型 calculated	其他健康檢查的狀態，最多 255 個	三台裡至少兩台活著才算健康
CloudWatch 警報	一條指標的資料流	拿內部指標當判準

字串比對：回應前 5,120 個位元組要出現指定字串，抓得到「活著但回錯誤頁」的假健康。

Route 53 挑記錄的四條規則

故障切換是路由策略加健康檢查：用 Failover 是主備，用加權、延遲多半是多活

規則	內容
一	沒掛健康檢查的記錄，永遠算健康
二	全部不健康時，全部算健康，照策略挑
三	Secondary 沒掛檢查，Primary 一壞就盲切過去
四	Primary 與 Secondary 都不健康，回 Primary

Alias 指 ALB 時不另建檢查，把 **Evaluate Target Health**（評估目標健康）設成 Yes；指 CloudFront 時不能開。

最壞斷線時間：判定時間加 TTL

講師補充的簡化估算，文件沒有；忽略了檢查器不同步與逾時

1 判定時間：間隔乘以失敗門檻

30 秒乘 3 次，約 90 秒才判定不健康。

2 TTL：判定前拿到的舊答案還要活多久

第 89 秒查到的答案，TTL 300 就活到第 389 秒。

3 到期後，還要下一次查詢與連線

正在失敗的連線不會自己搬到備援。

官方建議掛健康檢查的記錄 TTL 設 **60 秒以內**；跨區域的標準長相是東京 ALB 當 Primary、維修頁 S3 桶當 Secondary。

動手做：看 **Create health check** 表單裡的時間與門檻

第 3 步那兩個欄位就是斷線時間公式的前半段，間隔乘以門檻；第 4 步的 Health checker regions 決定哪些地區的檢查器參與投票，18% 算的是檢查器的數目，不是地區數。看完表單，本課的數字都有了落點

小步	做什麼	做完應該看到
1	Route 53 主控台左邊選單 Health checks → Create health check	表單第一區問你要監控什麼：Endpoint、Status of other health checks、State of CloudWatch alarm 三選一
2	留 Endpoint，看 Specify endpoint by 的兩個選項	IP address 與 Domain name；Protocol 可選 HTTP、HTTPS、TCP
3	展開 Advanced configuration	Request interval 有 Standard (30 seconds) 與 Fast (10 seconds)；Failure threshold 有一個數字欄位，本課時間線用的 3 是範例值，畫面上的預設值以你看到的為準
4	同一區往下看	String matching、Latency graphs、Invert health check status、Health checker regions 幾個選項，最後那個就是圖裡「世界各地的檢查器」
5	按 Cancel 離開	回到 Health checks 列表，沒有多出任何一筆

QUIZ 03 · CLF-C02

一家公司希望其 **Amazon EC2** 實例在高度可用的環境中運行，即使某個地理區域發生自然災害也能持續運作。哪一種方法可以達到此目標？

- ☐ A 在多個 AWS 區域使用 EC2 實例 (Use EC2 instances in multiple AWS Regions)
- ☐ B 在多個 Amazon CloudFront 位置使用 EC2 實例 (Use EC2 instances in multiple Amazon CloudFront locations)
- ☐ C 在多個邊緣位置使用 EC2 實例 (Use EC2 instances in multiple edge locations)
- ☐ D 在 AWS Local Zones 使用 EC2 實例 (Use EC2 instances in AWS Local Zones)

想 30 秒，等一下翻下一頁對答案。

QUIZ 03 · CLF-C02 · ANSWER

一家公司希望其 **Amazon EC2** 實例在高度可用的環境中運行，即使某個地理區域發生自然災害也能持續運作。哪一種方法可以達到此目標？

- A** 在多個 AWS 區域使用 EC2 實例 (Use EC2 instances in multiple AWS Regions)
- B 在多個 Amazon CloudFront 位置使用 EC2 實例 (Use EC2 instances in multiple Amazon CloudFront locations)
- C 在多個邊緣位置使用 EC2 實例 (Use EC2 instances in multiple edge locations)
- D 在 AWS Local Zones 使用 EC2 實例 (Use EC2 instances in AWS Local Zones)

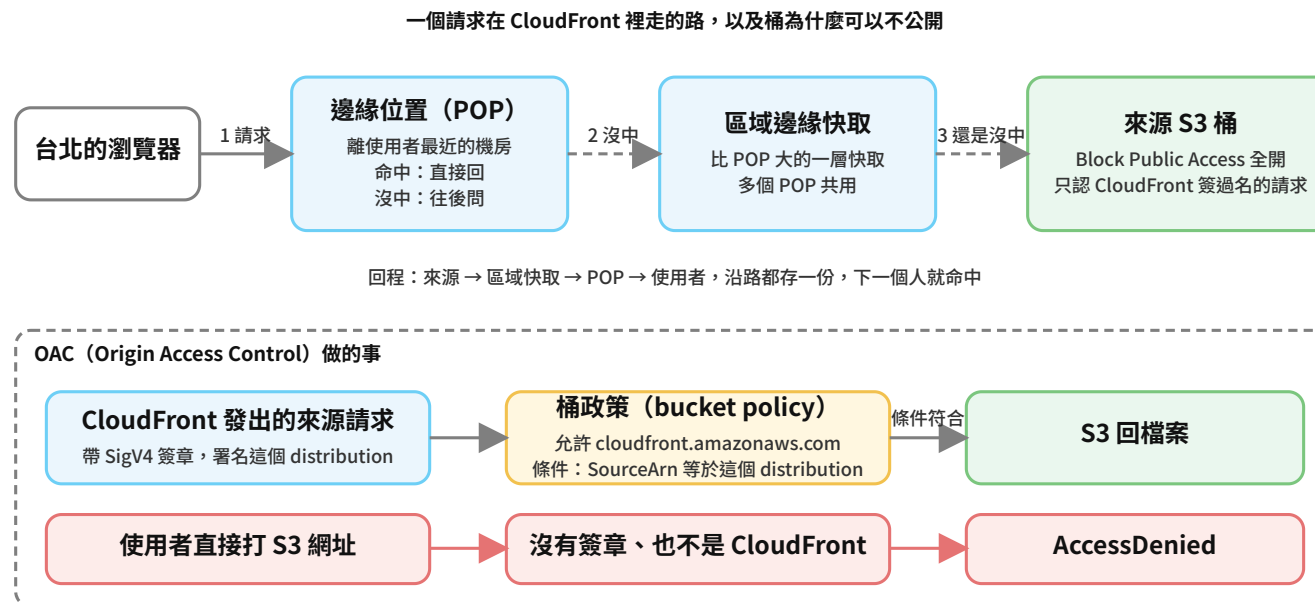
A。地區天災仍要運作，就得跨 Region 部署；CloudFront 位置與邊緣位置是快取點，不能跑 EC2；Local Zones 是某個 Region 延伸出去的小據點，不算另一個 Region。

快取放在門口，桶不用公開

- 一、邊緣位置命中就回，沒中才回來源拿
- 二、**OAC** 讓 **CloudFront** 簽名讀桶，桶維持不公開
- 三、改檔後邊緣不會自己知道：等 **TTL**、失效或換檔名

沒命中才往後問，回程沿路存一份

CloudFront：AWS 的內容分發網路，把檔案複製到全球邊緣位置，就近回給使用者



1 上排：邊緣位置沒中，先問區域邊緣快取，再沒中才回來源

2 區域邊緣快取：多個邊緣位置共用、容量更大的一層

3 下排：綠色是簽了名的請求被放行，紅色是直連 S3 被拒

建一個 **distribution**，網址只換網域

distribution（分發設定）：告訴 CloudFront 來源在哪、怎麼快取的一份設定

1 拿到一個 `dxxxx.cloudfront.net` 的名字

路徑照舊：`/img/logo.png` 換個網域就好。

2 檔案在邊緣預設放 24 小時

最短 0 秒、沒有上限；怎麼調看快取行為。

3 用自己的網域：三件事

填備用網域、掛 ACM 憑證、用 Alias 指過來。

ACM (AWS Certificate Manager)：免費簽發與續期 HTTPS 憑證的服務。

桶要不要公開？OAC 讓它不用

Block Public Access：S3 的四個總開關，擋住公開 ACL 與公開桶政策，新桶預設全開

做法	桶要不要公開	現在該不該用
桶設成公開讀	要，Block Public Access 得關	不該；任何人可繞過 CloudFront
OAI（舊版專用身分）	不用	舊專案才看到，官方建議搬到 OAC
OAC（Origin Access Control）	不用，四個開關全開	該；也支援 KMS 加密的物件

OAC 鎖的是入口：檔案仍可經 CloudFront 網址看到；要限制誰能看內容，另外用簽章 URL。

OAC 拆開看：CloudFront 簽名，S3 驗章

SigV4 簽章：用祕密金鑰把請求內容算成一串驗證碼，S3 重算一次，對得上才放行

1 CloudFront 這一半：Signing behavior 選 Sign requests

送往 S3 的每個請求都帶簽章，一律走 HTTPS。

2 S3 這一半：桶政策一條 Allow

Principal 是 `cloudfront.amazonaws.com`，Action 是 `s3:GetObject`。

3 Condition 的 `AWS:SourceArn` 是必要的

少了它，任何 distribution 都能讀你的桶。

主控台精靈選[建議設定](#)，會替你建 OAC 並把桶政策貼進去。

開了靜態網站託管，**OAC** 就用不上

× 大家通常會想

來源填 **s3-website** 那個網站端點，再加 **OAC**。

網站端點是 HTTP 的自訂來源，OAC 選項不會出現，桶就得公開。

✓ 實際上

來源填一般桶端點：桶名.s3.ap-northeast-1.amazonaws.com。

精靈按 Browse S3 選桶就會填成這種；網站端點也不支援 HTTPS。

CloudFront 是少數 **Always Free** 的服務

失效：你主動叫 CloudFront 把某些檔案從快取丟掉；快取自然到期不算失效

項目	免費額度
傳出到網際網路	每月前 1 TB
HTTP 與 HTTPS 請求	每月前 1,000 萬次
來源到 CloudFront 的傳輸	來源是 S3、ELB、API Gateway 時免費
失效 (invalidation)	每月前 1,000 條路徑

固定月費方案也有一級叫 **Free**；官方說 **Free Tier** 帳號不能訂，本課一律按量計費。

動手做：用 curl 看一個真實網站是不是從 CloudFront 回的

Via 標頭告訴你回應經過 CloudFront，X-Amz-Cf-Pop 告訴你是哪個邊緣位置

小步	做什麼	做完應該看到
1	Region 選 Asia Pacific (Tokyo)，打開 CloudShell	Bash 提示字元；接下來的請求是從這台在東京的 CloudShell 發出去的，不是從你桌上的電腦
2	打 <code>curl -sI https://d1.awsstatic.com/ grep -iE 'via x-cache x-amz-cf'</code>	幾行回應標頭（HTTP response header，伺服器隨網頁內容一起附上的幾行說明資訊，例如檔案型別、有沒有命中快取）； <code>curl -sI</code> 的大寫 I 是「只要標頭、不要內容」， <code>grep</code> 再從中撈出三行。Via 那行含 <code>cloudfront.net</code> 與 (CloudFront) 字樣；X-Amz-Cf-Pop 那行是一個代碼，講師經驗是它以機場代碼開頭、代表回應「發請求的那台機器」的邊緣位置，這裡是東京的 CloudShell，所以多半是 NRT 開頭；在你自己的電腦上跑，會換成離你近的
3	再打一次同樣的指令	X-Cache 那行顯示 Hit from cloudfront 或 Miss from cloudfront，兩次可能不同；Miss 只代表回應你的那個邊緣位置沒有，往後面拿了，標頭看不出是區域快取給的還是一路到來源（講師經驗）
4	把網址換成 <code>https://aws.amazon.com/</code> 再打一次	多半也看得到 Via 含 CloudFront，代表 AWS 自己的官網也走 CloudFront

QUIZ 04 · CLF-C02

哪一項 **AWS** 服務使用 **edge location** 來快取內容？

A

Amazon Kinesis

B

Amazon Simple Queue Service (Amazon SQS)

C

Amazon CloudFront

D

Amazon Route 53

想 30 秒，等一下翻下一頁對答案。

QUIZ 04 · CLF-C02 · ANSWER

哪一項 **AWS** 服務使用 **edge location** 來快取內容？

A Amazon Kinesis

B Amazon Simple Queue Service (Amazon SQS)

C Amazon CloudFront

D Amazon Route 53

C。「用 edge location 快取內容」就是 CloudFront 的定義；Route 53 也用邊緣位置但做的是 DNS 回應不是快取內容，Kinesis 與 SQS 跟邊緣無關。

快取的三個問題，三個名詞

快取行為（cache behavior）：符合某個路徑樣式的請求要怎麼處理的一組設定

一個網址進來：先比對路徑挑行為，再決定放多久，改檔後怎麼趕走舊的

第一步：路徑比對，由上往下，先中先贏

/api/* → 來源 B，CachingDisabled（不快取）

/images/*.jpg → 來源 A，放一年

*（預設行為，永遠最後）→ 來源 A，放一天

第二步：放多久，誰說了算

來源沒給 Cache-Control → 用 Default TTL（預設 24 小時）

來源給 max-age=3600 → 用它，但夾在 Min 與 Max 之間

Min TTL 大於 0 → 就算 no-cache 也至少放 Min 那麼久

第三步：檔案改了，邊緣還在發舊的，兩條路

失效（Invalidation）

對 distribution 送路徑，例如 /index.html 或 /*
所有邊緣幾秒內開始清，下一次請求回來源拿新的
每月前 1000 條路徑免費，/* 算一條
送出後不能取消

版本化檔名（官方建議）

app.css 改成 app.v2.css，HTML 改連結
新名字本來就沒快取，立刻生效
不用付失效的錢，也不怕瀏覽器或公司代理的舊快取
缺點：檔名與 HTML 連結要一起改

1 上排左：路徑比對由上往下，先中先贏，星號那筆永遠最後

2 上排右：放多久的三條規則

3 下排：改檔後兩條路，黃色超額要付錢，綠色是官方建議

路徑比對**先中先贏**，預設那筆墊底

每個行為綁一個路徑樣式、一個來源、一組快取與轉發的設定

1 預設行為的樣式固定是 *，改不了

接住所有沒被別的行為接走的請求。

2 其他行為的順序由你排

* 比對零到多個字元、? 剛好一個，大小寫有別。

3 順序排錯會出安全問題

/private/* 排在 *.mp4 後面，私人影片就被直接放行。

轉給來源，不等於分開存

Cookie：瀏覽器替網站記住的小紙條，登入後每個請求都自動帶上，所以能分辨 A 與 B

1 Cache policy：放多久、哪些請求算同一份

預設看網址，可再加查詢字串、標頭、Cookie。

2 Origin request policy：轉什麼給來源

轉過去的值不拿來區分快取。

3 登入 Cookie 只放後者，A 的回應可能回給 B

要分開存，就把它放進 Cache policy。

CloudFront 只快取 GET 與 HEAD 的回應（OPTIONS 可選）；POST 一律穿過去。

放多久：來源開價，政策定範圍

Cache-Control：來源回應的標頭，告訴快取這個檔可以放幾秒；TTL 三個值是政策裡的下限、預設、上限

情況	邊緣位置放多久
來源沒給 Cache-Control	Default TTL ；不掛政策時 24 小時
來源給 max-age=3600	用它，但夾在 Min 與 Max 之間
Min 大於 0，來源給 no-cache	照樣至少放 Minimum TTL
受管政策 CachingOptimized	Min 1 秒、Default 24 小時、Max 365 天
受管政策 CachingDisabled	三個都是 0，等於不快取

按強制重新整理，CloudFront 就回來源？

× 大家通常會想

瀏覽器帶 no-cache 上去，CloudFront 會回來源拿新的。

按再多次，邊緣那份還在 TTL 內就照樣回。

✓ 實際上

CloudFront 忽略瀏覽器請求裡的 Cache-Control 與 Pragma。

它只聽來源跟你的政策；過期後回來源問，沒變就回 304。

改了檔：救急用失效，常態換檔名

	失效 Invalidation	版本化檔名
做法	送一組路徑，例如 <code>/index.html</code> 或 <code>/*</code>	<code>app.css</code> 改名 <code>app.v2.css</code>
生效	幾秒內開始清，下次請求回來源	立刻，新名字沒有快取
費用	每月前 1,000 條路徑免費， <code>/*</code> 算一條	不用付失效的錢
瀏覽器與代理的快取	管不到	新網址，一定拿新的
缺點	送出後不能取消	HTML 裡的連結要一起改

`index.html` 這類固定網址的入口檔，TTL 設短或改版時失效它；**額度算路徑數**，不算檔案數。

HANDS-ON · 學員跟做 · 約 5 分鐘 · 0 元，只讀受管政策，不建任何東西

動手做：看兩個受管快取政策的 TTL 長什麼樣

本課那張 TTL 表就是這三筆政策的詳細頁，86400 秒等於 24 小時，31536000 秒等於 365 天；你之後替行為挑政策，就是在這幾筆裡選

小步	做什麼	做完應該看到
1	CloudFront 主控台左邊選單 Policies，停在 Cache 分頁	兩區：AWS managed 與 Custom，沒建過自訂政策的話 Custom 是空的
2	在 AWS managed 裡點 CachingOptimized	詳細頁顯示 Minimum TTL 1、Default TTL 86400、Maximum TTL 31536000，都是秒
3	回上一頁點 CachingDisabled	三個 TTL 都是 0
4	再點 UseOriginCacheControlHeaders	Minimum 0、Default 0、Maximum 31536000：來源說多久就多久
5	切到旁邊的 Response headers 分頁	有 SecurityHeadersPolicy 這一筆，第 8 課加分會用到

QUIZ 05 · CLF-C02

一家公司將其網站託管於 **Amazon EC2** 執行個體上。公司需要確保網站能夠接觸全球觀眾並提供最低延遲給使用者。公司應該使用哪一個 **AWS** 服務來滿足這些需求？

A

Amazon Route 53

B

Amazon CloudFront

C

彈性負載平衡 (Elastic Load Balancing)

D

AWS Lambda

想 30 秒，等一下翻下一頁對答案。

QUIZ 05 · CLF-C02 · ANSWER

一家公司將其網站託管於 **Amazon EC2** 執行個體上。公司需要確保網站能夠接觸全球觀眾並提供最低延遲給使用者。公司應該使用哪一個 **AWS** 服務來滿足這些需求？

A

Amazon Route 53

B

Amazon CloudFront

C

彈性負載平衡 (Elastic Load Balancing)

D

AWS Lambda

B。EC2 上的網站要觸及全球觀眾並給最低延遲，選 CloudFront；Route 53 只決定把人導到哪個端點，本身不加速內容傳輸；ELB 是單一區域內分流，Lambda 是執行程式。

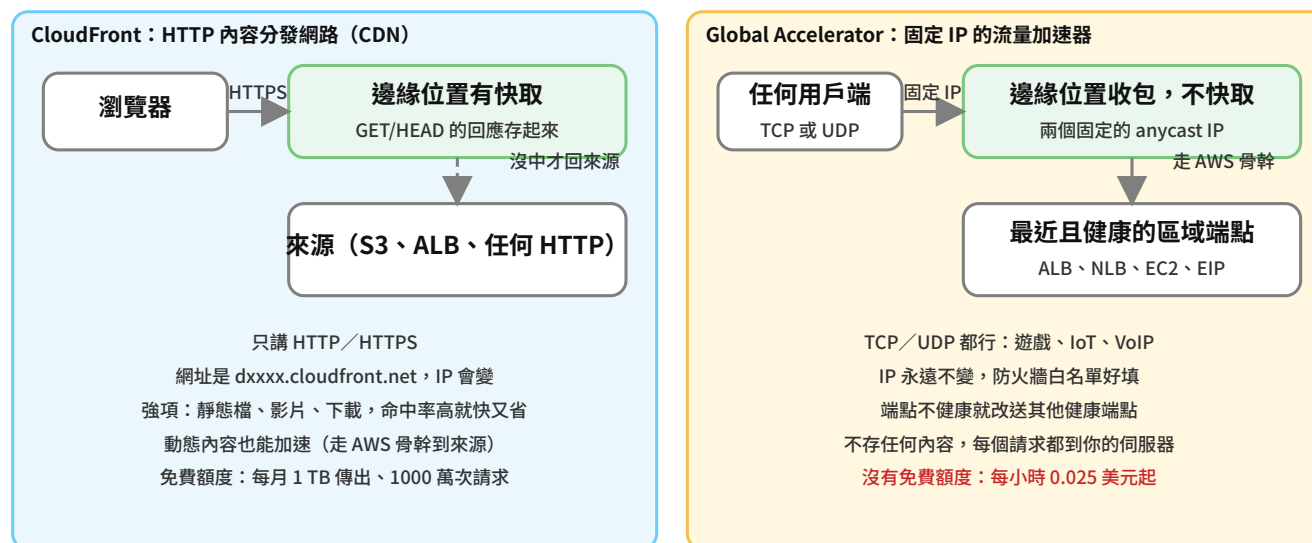
加速連線，與 **AI** 應用的邊界

- 一、**CloudFront** 加速內容，**Global Accelerator** 加速連線
- 二、延遲路由只導人，不會替你查模型在哪
- 三、聊天 **API** 是 **POST**，**CDN** 的快取價值在前端

兩個都住邊緣，一個存、一個不存

Global Accelerator：給你固定 IP，在邊緣接下連線，走 AWS 骨幹送到最近且健康的區域端點

兩個都用邊緣位置，做的事不一樣



- 1 左：CloudFront 有快取，沒中才回來源，只講 HTTP
- 2 右：Global Accelerator 只收包不存，TCP 與 UDP 都行
- 3 最下面紅字：一個有免費額度，一個按小時收費

固定 IP 加骨幹：盡早進 AWS 的網路

anycast IP：同一個 IP 在全球很多邊緣位置同時廣播，封包自然流到最近的那個

1 兩個靜態 anycast IP，來自兩個隔離的網路區塊

一個被擋或故障，用戶端改連另一個；白名單兩個都填。

2 端點：ALB、NLB、EC2 或 Elastic IP

端點不健康，新連線先送同區域其他健康端點。

3 兩種比例：端點權重與流量刻度盤

刻度盤調 50%，原本導向東京的流量只有一半進東京。

NLB（Network Load Balancer）只看 TCP 或 UDP 連線；accelerator 停用時 IP 還在、刪除就沒了。

選型：先問是不是 HTTP

UDP：不先建立連線就丟封包的協定；MQTT：物聯網（IoT）裝置常用的訊息協定；VoIP：網路電話

你的流量	選	為什麼
網站、圖片、影片、下載	CloudFront	可快取，命中率高就又快又省
HTTP 的動態 API	CloudFront 不快取行為，或 Global Accelerator	都走骨幹；要固定 IP 選後者
UDP 遊戲、IoT 的 MQTT、VoIP	Global Accelerator	不是 HTTP，CloudFront 收不了
客戶防火牆要 IP 白名單	Global Accelerator	兩個固定 IP；CloudFront 的 IP 會變
多區域主備，不靠 DNS 切換	Global Accelerator	端點健康一變就改送新連線

是 HTTP 再問能不能快取：能就 CloudFront；不能但要固定 IP 或快速區域切換，才找 Global Accelerator。

Global Accelerator 是更強的 CloudFront ？

× 大家通常會想

Global Accelerator 比較快，靜態網站也放它後面。

一萬個人要同一張圖，你的伺服器就被打一萬次。

✓ 實際上

它不存任何內容，每個請求都到來源。

CloudFront 把一萬次變一次；它對動態內容與 API 也有加速。

三個都能導流量，各管一層

Global Accelerator 偵測到端點不健康，約一分鐘內移出；已建立的連線不會被搬，新連線才改送

ALB

在一個區域裡，把流量分給後面的機器。

Route 53 故障切換

改 DNS 答案，受舊答案的 TTL 拖累。

Global Accelerator

在多個區域之間挑一個，同一個 IP 後面改送。

Global Accelerator 建了就跳表

項目	CloudFront	Global Accelerator
固定費	按量計費時沒有	每小時 0.025 美元，停用也照收
流量費	每月前 1 TB 傳出免費	標準傳輸費之上再加每 GB 加速費
免費額度	1 TB 傳出、1,000 萬次請求	沒有
掛著不動一個月	0 元	約 18 美元，加兩個 IPv4 約 25 美元

學員這一課只看建立畫面，[不按建立](#)；刪除才停止收費。

動手做：看 **Create accelerator** 表單的兩種類型與協定

第 3 步的 Protocol 選項就是這一課的分水嶺，CloudFront 的表單裡不會出現 UDP；第 4 步的 Region 與 Traffic dial 是「多區域之間挑一個」的具體長相

小步	做什麼	做完應該看到
1	主控台搜尋 Global Accelerator 進入，按 Create accelerator；名字填 a07-demo	下方 Accelerator type 有 Standard 與 Custom routing 兩種：Standard 由 Global Accelerator 替你挑最近且健康的端點，本課講的全是它；Custom routing 是讓你的程式自己決定每個使用者連到哪一台（例如遊戲配對把一群玩家送進同一台伺服器），本課不用
2	留 Standard，IP address type 留 IPv4	另一個選項 Dual-stack 是同時給 IPv4 與 IPv6，所以固定 IP 會變成兩個 IPv4 加兩個 IPv6 共四個
3	按 Next；Ports 填 80，Protocol 選 TCP（HTTP/1.1 與 HTTP/2 跑在 TCP 上，HTTP/3 走 UDP 這裡先不管；填這些只是為了能看下一頁）	這一頁叫 Add listener，Protocol 的選項有 TCP、UDP
4	再按 Next；Region 選 Asia Pacific (Tokyo)	這一頁叫 Add endpoint groups，有 Traffic dial 百分比欄位，預設 100
5	看完就停：不要再按 Next 進到 Add endpoints，直接離開頁面或按 Cancel	回到 Global Accelerator 首頁，列表沒有任何 accelerator

QUIZ 06 · CLF-C02

下列哪些 **AWS** 服務會使用全球邊緣位置？（選兩項）

- ☐ **A** AWS Fargate
- ☐ **B** Amazon CloudFront
- ☐ **C** AWS Global Accelerator
- ☐ **D** AWS Wavelength
- ☐ **E** Amazon VPC

想 30 秒，等一下翻下一頁對答案。

QUIZ 06 · CLF-C02 · ANSWER

下列哪些 **AWS** 服務會使用全球邊緣位置？（選兩項）

☐ A AWS Fargate

☒ B Amazon CloudFront

☒ C AWS Global Accelerator

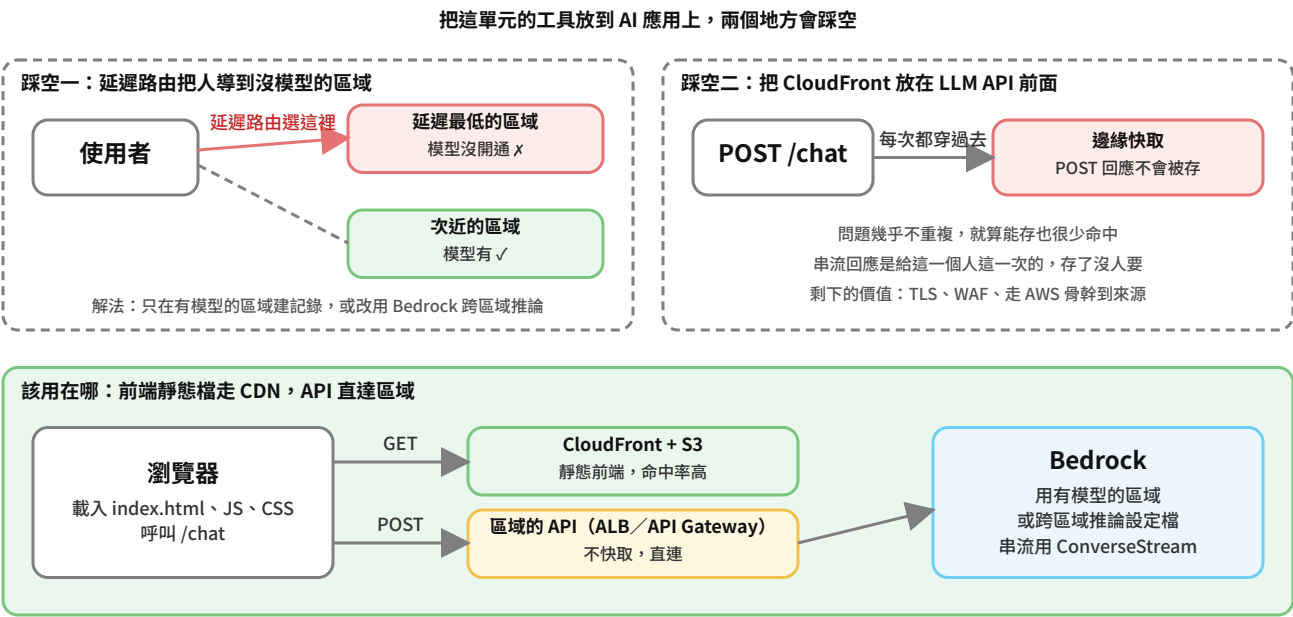
☐ D AWS Wavelength

☐ E Amazon VPC

B、C。用全球邊緣位置的服務是 CloudFront 與 Global Accelerator；Fargate 是容器運算、VPC 是區域內的網路、Wavelength 是把運算放進電信商的 5G 機房，都不是邊緣位置。

網站工具套到聊天機器人，踩空兩次

Bedrock：AWS 的模型服務；使用者呼叫你的聊天 API，你的 API 再呼叫 Bedrock



- 1 左上：延遲最低的區域模型沒開通，紅色箭頭是被導錯的那步
- 2 右上：POST 的回應不會被邊緣存下來
- 3 下排：前端走 CloudFront，API 直達區域，模型走跨區域推論

延遲路由不會替你查模型

× 大家通常會想

API 部署在東京與維吉尼亞，延遲路由把人導到最近的區域就好。

各區域能用的模型不一樣，官方是一個模型一張區域表。

✓ 實際上

被導到沒模型的區域，使用者拿到錯誤，不是低延遲。

延遲路由只看網路；這假設 API 呼叫同區域的 Bedrock。

兩層分開：DNS 導人，設定檔導請求

跨區域推論（cross-Region inference）：呼叫推論設定檔，Bedrock 在它允許的區域裡挑一個處理

1 死路：只在有模型的區域建延遲記錄

換模型時重新確認，不相容就得改 DNS。

2 活路：呼叫跨區域推論設定檔

目的區域本來就有模型；路由不另收費，走 AWS 內網。

3 上線前確認三件事

來源區域在支援清單、IAM 允許所有目的區域、SCP 沒擋。

SCP（Service Control Policy）：AWS Organizations 給整個帳號設的權限上限，能一刀禁掉某些區域。

有法規要求，選地理型

資料落地（data residency）：提示與回答在哪個地區被處理

設定檔	在哪些區域挑	資料落地
地理型	美國、歐洲、亞太這類範圍內	保證留在那個地理範圍
全球型	所有支援它的商業區域	不保證；SCP 要放行區域未指定的請求

東京沒有某模型的 In-Region（請求不出該區域），只要東京是亞太設定檔的來源區域，請求就送到亞太其他有模型的區域。

拆解「**CDN 加速 LLM**」：三刀

串流 (streaming)：模型每生成一小段就先送一段，使用者看到字一個一個出現

1 方法：聊天是 POST

POST 不快取，連區域邊緣快取都不經過。

2 內容：問題幾乎每次不同，回答帶隨機性

命中率本來就低，命中了舊答案也未必是你要的。

3 形狀：串流回應為這個人這一次生成

快取要嘛沒作用、要嘛回錯人的東西。

CloudFront 對聊天 API 剩下的是門：終止 HTTPS、掛 WAF (Web 應用程式防火牆)、走骨幹；加速不等於快取。

該放哪：前端走 **CDN**，**API** 直達

流量	放哪	用這單元的哪個工具
index.html 、 JS 、 CSS 、圖片	S3 加 CloudFront，OAC 鎖桶	快取命中率高
POST /chat	區域內的 ALB 或 API Gateway	直連，或經 CachingDisabled 行為
模型呼叫	有模型的區域，或跨區域推論	延遲路由只導人，不導模型
自己的網域	Alias 指到 CloudFront 與 ALB	www 與 api 用不同名字分開

想省模型費用，找應用層的**語意快取**：由你的程式判斷哪類問題能重用舊答案。

“

CDN 幫你把不會變的東西放到門口；
LLM 的回答為每個人各自生成，
快取價值幾乎都在前端。

—— 一句話帶走 ——

動手做：數兩個區域各有幾個模型

第 2、3 步的總數只是暖身，兩區數量相同清單也可能不同；真正接回本課的是第 4 步，同一個模型一區有、一區沒有，延遲路由把人導到沒有的那區就踩空，路由之前先查清單

小步	做什麼	做完應該看到
1	Region 選 Asia Pacific (Tokyo)，打開 CloudShell	Bash 提示字元
2	打 <code>aws bedrock list-foundation-models --region ap-northeast-1 --query 'length(modelSummaries)'</code>	一個數字，東京可用的模型數
3	把 <code>--region</code> 改成 <code>us-east-1</code> 再打一次	另一個數字，多半比東京大
4	挑一個你打算用的模型關鍵字（例如 nova），打 <code>aws bedrock list-foundation-models --region ap-northeast-1 --query 'modelSummaries[?contains(modelId, `nova`)].modelId' --output text tr '\t' '\n'</code> ，再把 <code>--region</code> 換成 <code>us-east-1</code> 打一次	兩張清單；某個 <code>modelId</code> 只出現在其中一區，就是「這個模型那一區沒有」的實證

QUIZ 07 · CLF-C02

以下哪些敘述正確描述了 **AWS** 全球基礎架構各元件之間的關係？（選兩項）

- ☐ **A** AWS 區域 (Regions) 的數量多於可用區域 (Availability Zones)
- ☐ **B** 邊緣節點 (edge locations) 的數量多於 AWS 區域 (Regions)
- ☐ **C** 邊緣節點 (edge location) 就是一個可用區域 (Availability Zone)
- ☐ **D** AWS 區域 (Regions) 的數量多於邊緣節點 (edge locations)
- ☐ **E** 可用區域 (Availability Zones) 的數量多於 AWS 區域 (Regions)

想 30 秒，等一下翻下一頁對答案。

QUIZ 07 · CLF-C02 · ANSWER

以下哪些敘述正確描述了 **AWS** 全球基礎架構各元件之間的關係？（選兩項）

- ☐ A AWS 區域 (Regions) 的數量多於可用區域 (Availability Zones)
- ☒ B 邊緣節點 (edge locations) 的數量多於 AWS 區域 (Regions)
- ☐ C 邊緣節點 (edge location) 就是一個可用區域 (Availability Zone)
- ☐ D AWS 區域 (Regions) 的數量多於邊緣節點 (edge locations)
- ☒ E 可用區域 (Availability Zones) 的數量多於 AWS 區域 (Regions)

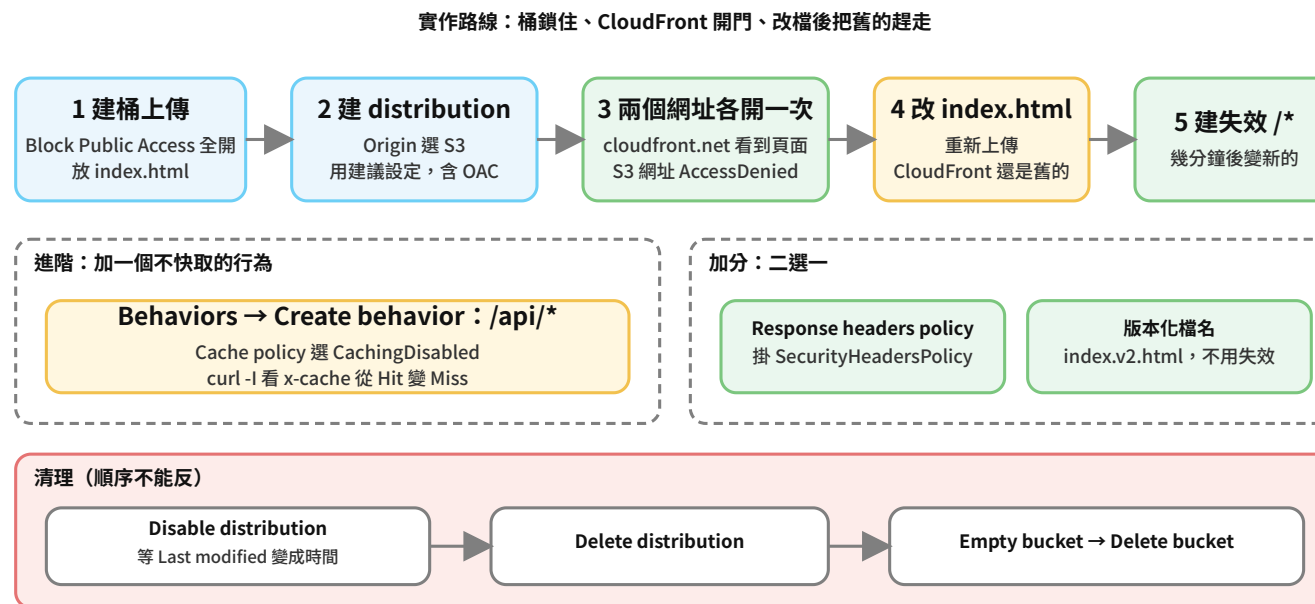
B、E。數量上邊緣位置最多、可用區其次、區域最少；C 錯在邊緣位置是做快取、DNS 與邊緣運算的機房，不能開 EC2。邊緣位置很多，但每個模型只在自己清單上的區域開通。

實作：S3 靜態前端掛上 CloudFront

- 一、桶從頭到尾不公開，匿名只能走 **cloudfront.net**
- 二、改檔之後邊緣不會自己知道，送失效才換新
- 三、清理要兩段：先 **Disable**，等部署完再 **Delete**

實作：一個鎖著的桶、兩個網址

零程式碼，全部在主控台點；前面的實作沒做過也做得完



- 1 上排五格是必做主線，第三格是主角：一個網址能開、一個不能
- 2 中排左邊是進階，右邊是加分
- 3 下排紅框是清理，照官方建議的順序走

三層：必做八步就及格

先講錢：CloudFront 與失效都在免費額度內，整份實作預期 0 元，做完當天刪掉

必做 70%

八步照做一定過：建桶、建 **distribution**、兩個網址、失效。

進階 20%

加一條 **/api/*** 不快取的行為，用 **curl** 看 Hit 與 Miss。

加分 10%

掛安全標頭，或用版本化檔名不做失效。

實作：開工前的準備

用第一單元建的 course-admin 登入；全程在 Tokyo 區域

1

主控台右上角區域切到 **Asia Pacific (Tokyo)**

2

在自己電腦用任何文字編輯器建 **index.html**，內容只有一行

```
<h1>v1 hello from S3 and CloudFront</h1>
```

等一下改檔時把 v1 改成 v2。

必做：八步 (1/3)

步	做什麼	做完應該看到
1 建桶	S3 → General purpose buckets → Create bucket ; Bucket name 填 a07-web-加你的英文名或幾個亂數 (全小寫、全球唯一) ; Region 是 Tokyo ; Object Ownership 留 ACLs disabled ; Block Public Access settings 四個勾全部保留 ; 其他預設 ; 按 Create bucket	桶列表多一個，Access 欄顯示 Bucket and objects not public
2 上傳	點進桶 → Upload → Add files 選 index.html → Upload	Upload: status 頁顯示 Succeeded
3 直接打 S3	在瀏覽器開 <code>https://你的桶名.s3.ap-northeast-1.amazonaws.com/index.html</code>	一段 XML，Code 是 AccessDenied ; 桶沒公開，本來就該這樣
4 建 distribution	CloudFront → Distributions → Create distribution ; Distribution name 填 a07-web ; 選 Single website or app → Next ; Domain setup 跳過 → Next ; Origin type 選 Amazon S3 , S3 origin 按 Browse S3 選你的桶 ; Settings 選 Use recommended origin settings → Next	精靈說明裡提到會替你設定 Origin Access Control 並更新桶政策

必做：八步 (2/3)

步	做什麼	做完應該看到
5 不開 WAF	Enable security protections 頁選不啟用 → Next → Create distribution	回到 distribution 頁，Last modified 顯示 Deploying
6 等部署	等 Last modified 從 Deploying 變成一個日期時間，通常幾分鐘；複製 General 分頁的 Distribution domain name	一個 dxxxxxxxx.cloudfront.net 的名字
7 兩個網址各開一次	瀏覽器開 https://那個名字/index.html；再回頭開第 3 步的 S3 網址；然後到 S3 桶 → Permissions → Bucket policy	CloudFront 網址看到 v1 的標題；S3 網址仍是 AccessDenied；桶政策多了一條 Principal 是 cloudfront.amazonaws.com、Condition 是 AWS:SourceArn 的 Allow，那個 ARN 最後一段就是你 distribution 頁上方的 Distribution ID

必做：八步 (3/3)

步	做什麼	做完應該看到
8 改檔與失效	把 index.html 的 v1 改成 v2，重做第 2 步覆蓋上傳；重新整理 CloudFront 網址，多半還是 v1；到 distribution → Invalidations → Create invalidation，Object paths 填 /* → Create invalidation；等 Status 變 Completed 再重新整理	失效前 v1、失效後 v2

改成 v2 還是看到 v1：邊緣那份還沒過期

建議設定給的是 CachingOptimized，Default TTL 24 小時，從邊緣第一次存下這份起算

1 重新整理帶的 no-cache，CloudFront 不理

按再多次，也不會逼它回 S3 拿。

2 送 /* 失效，幾秒內所有邊緣開始清

/* 只算一條路徑，在每月 1,000 條免費額度內。

3 冷門檔案可能提早被清，直接看到 v2 也正常

照樣送失效，Status 到 Completed 就算完成。

進階與加分 (1/2)

層	做什麼	看到什麼算成功
進階	再建一個檔 ping.json 內容 {"ok":true}；到桶裡按 Create folder 取名 api，點進 api 資料夾再 Upload → Add files 選 ping.json → Upload；到 distribution → Behaviors → Create behavior，加一條行為 (cache behavior，依網址路徑套用不同處理規則，第 5 課)：Path pattern 填 /api/*，Origin 選同一個桶，Cache policy 選 CachingDisabled，其他預設 → Create behavior；等部署完，打開 CloudShell (主控台內建的瀏覽器終端機，上方搜尋列旁的終端機圖示，第 1 課動手做用過) 打兩次 <code>curl -sI https://那個名字/index.html grep -iE 'HTTP/ x-cache'</code>，再打兩次 <code>curl -sI https://那個名字/api/ping.json grep -iE 'HTTP/ x-cache'</code>	index.html 第二次是 Hit from cloudfront (邊緣位置直接回)；ping.json 兩次都是 Miss from cloudfront (邊緣沒有、往後拿了，不是失敗，狀態碼仍是 200)，因為那條行為不快取

進階與加分 (2/2)

層	做什麼	看到什麼算成功
加分一	distribution → Behaviors → 勾 Default (*) → Edit → Response headers policy 選 SecurityHeadersPolicy → Save changes；等部署完 <code>curl -sI https://那個名字/index.html</code>	回應多了 <code>x-content-type-options: nosniff</code> (要瀏覽器不要猜檔案型別)、 <code>x-frame-options: SAMEORIGIN</code> (不准別的網站把你的頁面嵌進 iframe)、 <code>strict-transport-security</code> (要瀏覽器之後只用 HTTPS) 等標頭
加分二	把 index.html 內容改成 v3，另存成 index-v3.html 上傳；開 <code>https://那個名字/index-v3.html</code>	立刻是 v3，不用失效，因為新名字本來就沒被快取

卡住了：提示卡

卡在哪	怎麼辦
建桶說名字已被使用	桶名全球唯一，換一組亂數
直接打 S3 是 NoSuchKey	檔名大小寫不對，或上傳沒成功
找不到 Use recommended origin settings	手動選 OAC、Create new OAC，複製政策貼到桶
CloudFront 網址回 AccessDenied	看桶政策有沒有 <code>cloudfront.amazonaws.com</code> 那條
失效完還是 v1	換無痕視窗，或在 CloudShell 用 curl 看

建 distribution 時不要啟用 WAF：那會建一個另外收費的 web ACL。

清理

順序	做什麼	做完應該看到
1	CloudFront → Distributions → 勾 a07-web → Disable → 確認	Status 變 Disabled，Last modified 又變 Deploying
2	等 Last modified 變回日期時間，再勾它 → Delete → 確認	列表沒有它；Delete 按不下去就是還在部署，再等
3	S3 → General purpose buckets → 勾你的桶 → Empty → 輸入桶名 → Empty	Empty bucket: Status 頁顯示完成
4	再勾同一個桶 → Delete → 輸入桶名 → Delete bucket	桶列表沒有它
5	選作：CloudFront 左邊 Origin access 看那個自動建的 OAC，沒有別的 distribution 在用就可以刪	定價頁沒有 OAC 這個收費項目（講師經驗），留著也沒關係

實作繳交：三張截圖加一句話

個人完成；只交必做就及格，刪資源之前先把截圖拍好

- 1 CloudFront 網址顯示 v1 或 v2 的頁面
- 2 S3 網址的 AccessDenied
- 3 失效的 Status 是 Completed
- 4 一句話寫失效前後各看到什麼；做了進階附兩組 curl 輸出

驗收 必做 70%（三張截圖各 20%、那一句 10%）；進階 Hit 與兩次 Miss 20%；加分 10%。

QUIZ 08 · CLF-C02

根據 **AWS shared responsibility model**（**AWS 共同責任模型**），下列哪一項是客戶的責任？

A

為 Amazon S3 基礎設施裝置套用安全修補程式

B

提供 AWS 資料中心的實體安全

C

為 Lambda@Edge 安裝作業系統更新

D

為 IAM 使用者帳戶實作多重驗證 (MFA)

想 30 秒，等一下翻下一頁對答案。

根據 **AWS shared responsibility model**（**AWS 共同責任模型**），下列哪一項是客戶的責任？

A 為 Amazon S3 基礎設施裝置套用安全修補程式

B 提供 AWS 資料中心的實體安全

C 為 Lambda@Edge 安裝作業系統更新

D 為 IAM 使用者帳戶實作多重驗證 (MFA)

D。為 IAM 使用者實作 MFA 是客戶的；A 的 S3 基礎設施裝置修補、B 的資料中心實體安全、C 的 Lambda@Edge 作業系統更新都是 AWS 的責任，Lambda@Edge 是在邊緣位置跑程式的服務，底層機器 AWS 顧。

回家作業：都拿期末專題來做，一律個人完成

每一份都接著上一份做，最後得到專題的整條入口設計

DNS 記錄表

根網域、www、api 各指到什麼、型別、TTL、理由。

選路由策略

兩區域求快、單區域給新版 10%，各寫策略與限制。

一組故障切換

Primary、Secondary、TTL，加一條最壞斷線算式。

前端送檔路線

使用者、邊緣位置、來源桶三框，標出 OAC 在哪段。

行為表

靜態、API、預設三列：路徑、來源、快取政策、理由。

選邊與拆解

三種流量選哪一邊；再讓 AI 追問「CDN 加速 LLM」三輪。

TAKEAWAY

- 01** IP 會變的 AWS 資源接名字用 **Alias**，根網域尤其如此。
- 02** 故障切換是路由策略加健康檢查；最壞斷線是**判定時間加 TTL**。
- 03** CloudFront 是門口的快取，**OAC 讓桶不用公開**；救急用失效，常態換檔名。
- 04** Global Accelerator 加速連線不存內容；聊天 API 是 POST，**CDN 的快取價值在前端**。

Reference

- **Route 53 : Supported DNS record types** [三種記錄](#)
- **Route 53 : Choosing between alias and non-alias records** [Alias 與根網域](#)
- **Route 53 : Choosing a routing policy ; Route 53 Pricing** [路由策略與查詢費](#)
- **Route 53 : How Route 53 chooses records when health checking is configured** [故障切換](#)
- **CloudFront : Restrict access to an Amazon S3 origin** [OAC 鎖桶](#)
- **CloudFront : Cache behavior settings ; Invalidate files** [行為與失效](#)
- **Global Accelerator : How it works ; FAQs** [加速連線與選型](#)
- **Amazon Bedrock : Regional availability by models ; cross-Region inference** [模型區域](#)